

C Language — Cheatsheet com Exercícios (pythontutor.com)

Exercícios:

<https://claude.ai/public/artifacts/f99f6ab2-60aa-4d8e-8cf4-ddb24015076a>

Site usado para realizá-los:

<https://pythontutor.com/c.html>



Exercício 2 — Inspeccionando tamanhos

C

```
#include <stdio.h>
```

```
int main(void) {  
    printf("char:  %zu byte(s)\n", sizeof(char));  
    printf("int:   %zu byte(s)\n", sizeof(int));  
    printf("long:  %zu byte(s)\n", sizeof(long));  
    printf("float: %zu byte(s)\n", sizeof(float));  
    printf("double: %zu byte(s)\n", sizeof(double));  
    return 0;  
}
```

O que observar: os valores impressos confirmam a tabela acima.

Valores impressos com o código acima:

```
char:  1 byte(s)  
int:   4 byte(s)  
long:  8 byte(s)  
float: 4 byte(s)  
double: 8 byte(s)
```

Tente mudar `sizeof(int)` para `sizeof(short)` e compare.

```
char:  1 byte(s)  
int:   2 byte(s)  
long:  8 byte(s)  
float: 4 byte(s)  
double: 8 byte(s)
```

O modificador de tamanho (short), muda o número de bytes (espaço) que uma variável ocupa na memória, encolhendo-o para economizar memória (cortou o int pela metade).



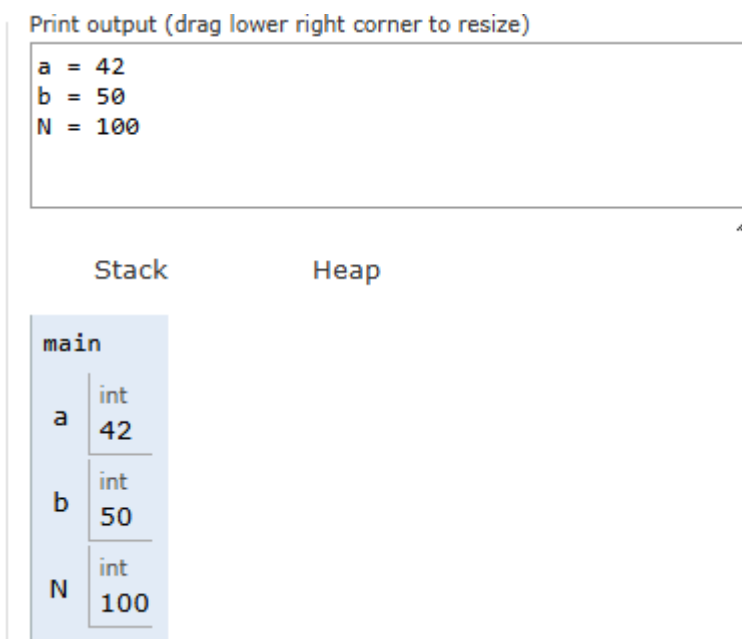
Exercício 3 — Variável não inicializada vs. inicializada

C

```
#include <stdio.h>
```

```
int main(void) {  
    int a = 42;  
    int b;      /* valor lixo — NÃO leia antes de atribuir */  
    b = a + 8;  
    printf("a = %d\n", a);  
    printf("b = %d\n", b);  
  
    const int N = 100;  
    /* N = 200; */ /* descomente para ver o erro de compilacao */  
    printf("N = %d\n", N);  
    return 0;  
}
```

Valor impresso do código acima:



O que observar: o valor de **a** aparece imediatamente no frame; **b** só recebe valor após a atribuição.

Tente descomentar **N = 200** e veja o erro.

error: assignment of read-only variable 'N'

Isso acontece quando se usa a palavra-chave `const` antes do tipo de uma variável, isso torna aquele valor constante (imutável), sendo assim inalterável após a sua inicialização.



Exercício 4 — Divisão inteira, módulo e incremento

C

```
#include <stdio.h>
```

```
int main(void) {  
    int a = 17, b = 5;  
  
    printf("%d / %d = %d\n", a, b, a / b); /* divisao inteira */  
    printf("%d %% %d = %d\n", a, b, a % b); /* resto */  
  
    int x = 3;  
    int pre = ++x; /* x vira 4, pre = 4 */  
    int pos = x++; /* pos = 4, x vira 5 */  
    printf("pre=%d pos=%d x=%d\n", pre, pos, x);  
  
    int max = (a > b) ? a : b;  
    printf("max(%d,%d) = %d\n", a, b, max);  
    return 0;  
}
```

O que observar: diferença entre pré e pós-incremento no valor retornado; resultado da divisão inteira vs. real.

Retorna:

Print output (drag lower right corner to resize)

```
17 / 5 = 3
17 % 5 = 2
pre=4 pos=4 x=5
max(17,5) = 17
```

Stack

Heap

main	
a	int 17
b	int 5
x	int 5
pre	int 4
pos	int 4
max	int 17

Pré-incremento (++x): Primeiro o código **soma 1** à variável, e depois entrega o valor atualizado para a expressão.

Pós-incremento (x++): Primeiro o código **entrega o valor atual** da variável para a expressão, e só depois soma 1 a ela.



Exercício 5a — if / else if / else

c

```
#include <stdio.h>
```

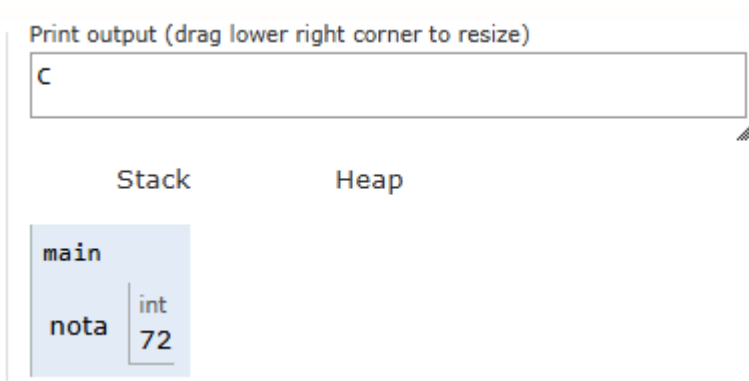
```
int main(void) {
    int nota = 72;

    if (nota >= 90) printf("A\n");
    else if (nota >= 80) printf("B\n");
    else if (nota >= 70) printf("C\n");
    else printf("Reprovado\n");

    return 0;
}
```

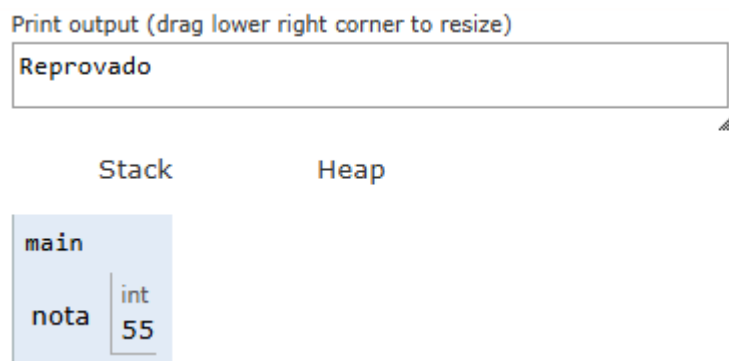
Valores retornados:

Código sem alterações



Variação: troque **nota** por 55, 80 e 95; execute cada versão e acompanhe qual ramo é percorrido.

Código com a nota = 55



Código com a nota = 80



Código com a nota = 90



Exercício 5b — for com break e continue

C

```
#include <stdio.h>
```

```
int main(void) {
    for (int i = 0; i < 10; i++) {
        if (i % 2 == 0) continue; /* pula pares */
        if (i == 7) break; /* para no 7 */
        printf("%d\n", i);
    }
    return 0;
}
```

O que observar: o valor de `i` no frame a cada passo; quando `continue` pula a impressão e quando `break` encerra o laço.

O laço for testa a variável `i` de 0 a 9. Quando `i` é par (0, 2, 4, 6, ou seja, o resto da divisão por 2 é igual a zero), o comando `continue` ignora o `printf` e pula para o próximo número. Quando `i` chega a 7, o `break` encerra o laço imediatamente, impedindo que os números 7, 8 e 9 sejam processados. Apenas 1, 3 e 5 (ímpares) são impressos.

Exercício 5c — do-while

C

```
#include <stdio.h>
```

```
int main(void) {
    int n = 1;
    do {
        printf("n = %d\n", n);
        n *= 2;
    } while (n < 32);
    return 0;
}
```

O que observar: o corpo executa *antes* do teste; **n** dobra a cada iteração (potências de 2).

O que o código abaixo mostra quando executado:

```
n = 1
n = 2
n = 4
n = 8
n = 16
```

Stack

Heap

```
main
├── int
│   └── 32
└── n
```

O corpo dobra o **n** antes do teste, pois o teste apenas determina se a variável atende aos critérios da condição estabelecida no final do loop. Caso seja afirmativo, o loop roda mais uma vez, como não foi, a variável permaneceu com o valor dobrado, sem ser impresso na tela.

Exercício 5d — switch com fall-through

c

```
#include <stdio.h>
```

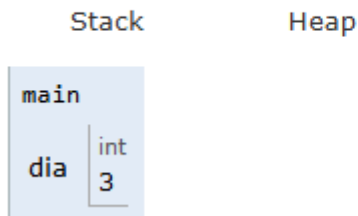
```
int main(void) {
    int dia = 3; /* 1=Seg ... 7=Dom */

    switch (dia) {
        case 1: case 2: case 3: case 4: case 5:
            printf("Dia util\n");
            break;
        case 6: case 7:
            printf("Fim de semana\n");
            break;
        default:
            printf("Invalido\n");
    }
    return 0;
}
```

O que observar: o *fall-through* intencional agrupa casos; remova um **break** para ver o comportamento inesperado.

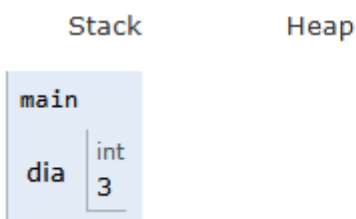
Saída com o break:

```
Dia util
```



Saída sem o primeiro break:

```
Dia util  
Fim de semana
```



Quando o programa encontra um case que corresponde ao valor da variável, ele entra ali e começa a executar o código. A partir desse ponto, o computador ignora todos os outros case seguintes e continua executando linha por linha, de cima para baixo, até que encontre um comando break ou o switch termine. Como o dia era 3 e se enquadrou no primeiro grupo, ele foi considerado válido, sem o break, ele executou linha por linha até o break mais próximo.

Exercício 6 — Pilha de chamadas em ação

c

```
#include <stdio.h>
```

```
int fatorial(int n) {  
    if (n <= 1) return 1;  
    return n * fatorial(n - 1); /* chamada recursiva */  
}
```

```
void dobra(int *x) { *x *= 2; }
```

```
int main(void) {
```



```

int f = fatorial(5);
printf("5! = %d\n", f);

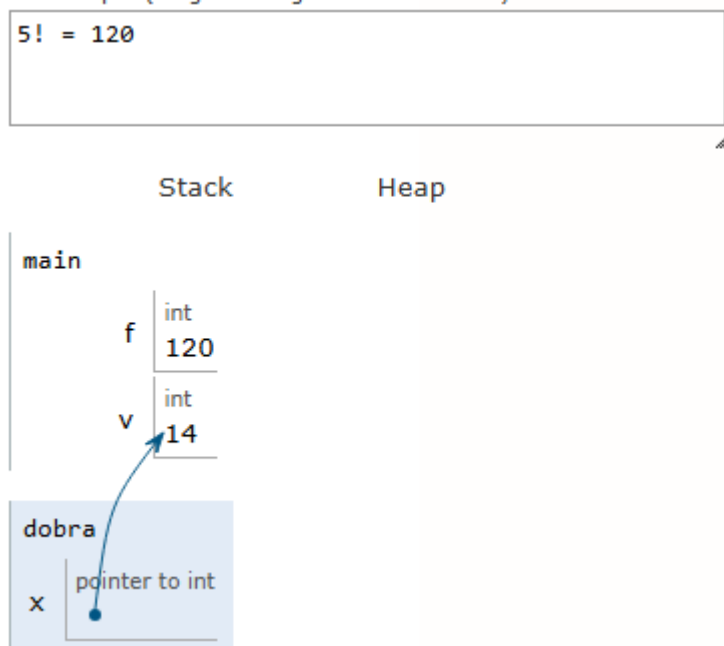
int v = 7;
dobra(&v);
printf("dobro de 7 = %d\n", v);
return 0;
}

```

O que observar: os frames de `fatorial` empilhando e desempilhando; como `dobra` modifica `v` via ponteiro — o frame de `dobra` mostra `x` apontando para `v` em `main`.

Na primeira parte, a função `fatorial` realiza chamadas recursivas, o que faz com que o sistema empilhe múltiplos frames na memória (de `fatorial(5)` até `fatorial(1)`), onde cada um aguarda o resultado do próximo. Ao atingir o caso base ($n \leq 1$), a pilha é desempilhada de trás para frente, resolvendo as multiplicações consecutivas até retornar o valor final de 120 para a função `main`.

Na segunda parte, a função `dobra` utiliza ponteiros para modificar o valor de uma variável local da `main`. Ao passar o endereço de memória (`&v`), o frame da função `dobra` cria um ponteiro `x` que aponta diretamente para o endereço de `v` no frame da `main`. A operação de desreferenciação (`*x *= 2`) altera o valor diretamente na origem, fazendo com que a variável `v` passe a valer 14 de forma permanente.



Saída final:

```
5! = 120
dobro de 7 = 14
```

Stack

Heap

main	
f	int
	120
v	int
	14

Exercício 7 — Array unidimensional e matriz

c

```
#include <stdio.h>
```

```
int main(void) {
    int v[5] = {10, 20, 30, 40, 50};

    /* soma dos elementos */
    int soma = 0;
    for (int i = 0; i < 5; i++)
        soma += v[i];
    printf("soma = %d\n", soma);

    /* matriz 2x3 */
    int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
    printf("m[1][2] = %d\n", m[1][2]); /* 6 */
    return 0;
}
```

O que observar: o PythonTutor mostra o array como células numeradas; acompanhe **soma** acumulando a cada iteração.

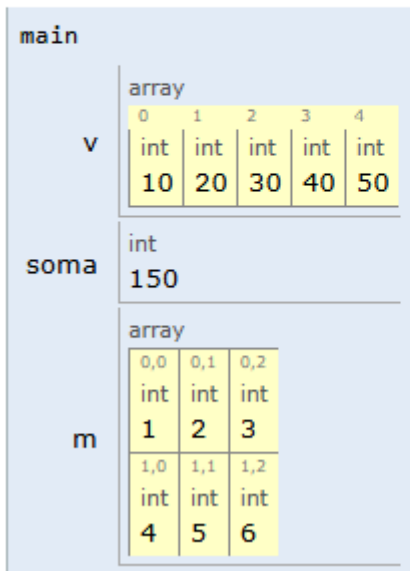
O vetor de 5 elementos é percorrido por um laço for, onde a variável soma armazena as somas de seu próprio valor + o valor de v[i] (soma += v[i]). Depois, é declarada uma matriz de duas linhas e três colunas, com os valores de 1 a 6. Por fim, é impresso o valor contido na segunda linha e terceira coluna, que corresponde ao número 6.

Saída final:

```
soma = 150
m[1][2] = 6
```

Stack

Heap



Exercício 8 — Percorrendo uma string caractere a caractere

c

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main(void) {
```

```
    char s[] = "Brasilia";
```

```
    int len = strlen(s);
```

```
    printf("Comprimento: %d\n", len);
```

```
    /* percorre e imprime cada char e seu codigo ASCII */
```

```
    for (int i = 0; i < len; i++)
```

```
        printf("s[%d] = '%c' (ASCII %d)\n", i, s[i], s[i]);
```

```
    /* comparacao */
```

```
    char t[] = "Brasilia";
```

```
    printf("strcmp: %d\n", strcmp(s, t)); /* 0 = iguais */
```

```
    return 0;
```

```
}
```

O que observar: o array `s` no frame de `main` com cada byte; o `'\0'` invisível no último índice (valor 0). Compare com `char *s = "Brasilia"` — o PythonTutor mostra `p` como uma seta para fora do frame (memória estática).

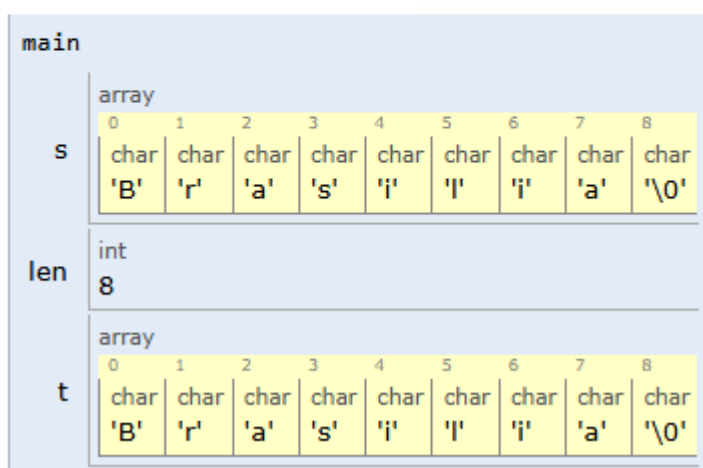
O código demonstra a manipulação de cadeias de caracteres (strings) na linguagem C. a string `s`, com o texto "Brasilia", e armazenada na memória como um vetor do tipo `char`. Este tipo de vetor finaliza a palavra com o caractere nulo (`\0`).

A variável `len` armazena o comprimento do texto retornado pela função `strlen(s)`. Depois, um laço `for` percorre esse vetor índice por índice, onde o `printf` exibe a letra literal (`%c`) e o seu respectivo código numérico decimal na tabela ASCII (`%d`). Por fim, é declarada uma segunda string `t` idêntica e o programa imprime o resultado da função `strcmp(s, t)`, que corresponde ao número 0 porque as duas strings são iguais.

Saída do código sem alteração:

```
s[4] = 'i' (ASCII 105)
s[5] = 'l' (ASCII 108)
s[6] = 'i' (ASCII 105)
s[7] = 'a' (ASCII 97)
strcmp: 0
```

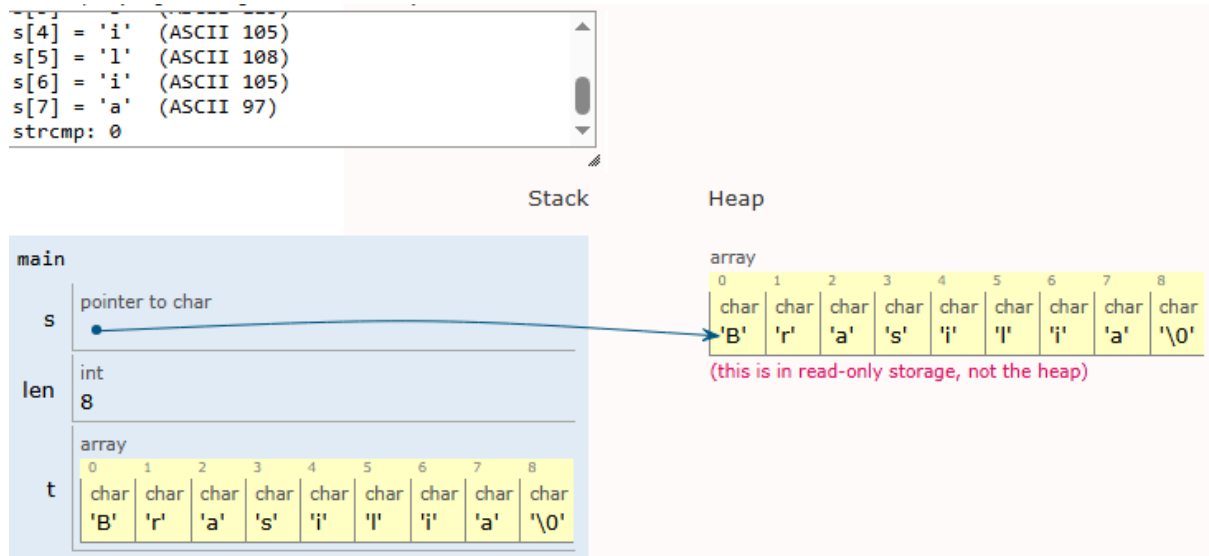
Stack



A diferença está em como a string é guardada na memória. No código anterior (`char s[]`), "Brasilia" é um vetor comum salvo dentro da função `main`, o que permite alterar as letras se necessário. Já no novo código (`char *p`), a palavra fica salva em uma área especial da memória que é de apenas leitura. A variável `p` é um ponteiro (uma seta) que apenas guarda o

endereço de onde essa palavra está. Por isso, qualquer tentativa de mudar alguma letra usando `p` fará o programa dar erro e fechar sozinho.

Saída do código com `char *s = "Brasilia":`



Exercício 9a — Ponteiro básico e de referência

c

```
#include <stdio.h>
```

```
int main(void) {
    int x = 10;
    int *p = &x;

    printf("x  = %d\n", x);
    printf("&x = %p\n", (void*)&x);
    printf("p  = %p\n", (void*)p);
    printf("*p = %d\n", *p);

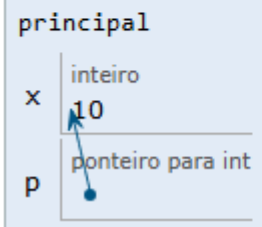
    *p = 99;
    printf("x apos *p=99: %d\n", x);
    return 0;
}
```

O que observar: no PythonTutor, `p` é exibido como uma **seta** apontando para a caixa de `x`. Após `*p = 99`, a caixa de `x` muda de valor sem que `x` apareça no lado esquerdo da atribuição.

```
x = 10
&x = 0xffff000bd4
p = 0xffff000bd4
```

Pilha

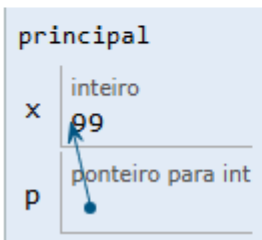
Monte



```
x = 10
&x = 0xffff000bd4
p = 0xffff000bd4
*p = 10
x apos *p=99: 99
```

Pilha

Monte



Ao fazer `p = 99`, o operador (desreferenciação) diz ao programa para ir até o endereço armazenado em `p` e modificar o valor guardado lá. Como esse endereço pertence a `x`, o valor na "caixa" de `x` é alterado diretamente na memória para 99, mesmo sem usarmos o nome da variável `x` na atribuição.



Exercício 9b — Aritmética de ponteiros e equivalência com índice

c

```
#include <stdio.h>
```

```
int main(void) {
    int v[4] = {10, 20, 30, 40};
    int *p = v;      /* p aponta para v[0] */

    printf("*p = %d\n", *p);    /* 10 */
}
```

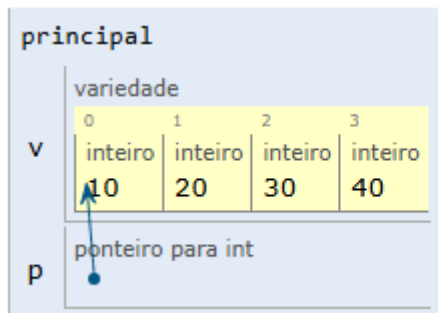
```

printf("*(p+1) = %d\n", *(p+1)); /* 20 */
printf("*(p+2) = %d\n", *(p+2)); /* 30 */

p++;          /* agora aponta para v[1] */
printf("apos p++: *p = %d\n", *p); /* 20 */
return 0;
}

```

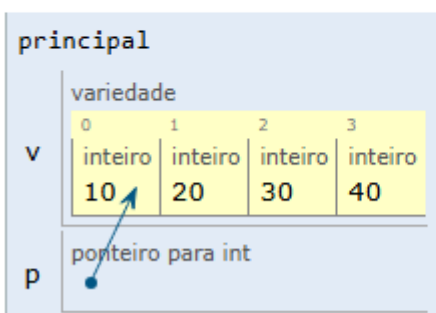
O que observar: a seta de **p** se mover entre os elementos do array a cada **p++**.



Em C, o nome de um array (**v**) funciona como um ponteiro para o seu primeiro elemento (**v[0]**).

Quando fazemos ***(p + 1)**, o compilador calcula o endereço do próximo elemento somando o tamanho de um **int** ao endereço atual, o que equivale exatamente a acessar **v[1]**. Ao executar

p++, o ponteiro em si é atualizado para guardar o endereço de memória do elemento seguinte, movendo a "seta" permanentemente para **v[1]**.



Exercício 9c — Ponteiro para ponteiro

c

```
#include <stdio.h>
```

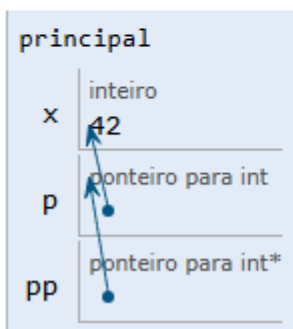
```
int main(void) {  
    int x = 42;  
    int *p = &x;  
    int **pp = &p;  
  
    printf("x = %d\n", x);  
    printf("*p = %d\n", *p);  
    printf("**pp = %d\n", **pp);  
  
    **pp = 100;    /* altera x via dois níveis de indireção */  
    printf("x apos **pp=100: %d\n", x);  
    return 0;  
}
```

O que observar: duas setas encadeadas — $pp \rightarrow p \rightarrow x$.

```
x  = 42  
*p  = 42  
**pp = 42
```

Pilha

Monte



O ponteiro pp é um ponteiro de segundo nível (int), o que significa que ele armazena o endereço de memória de outro ponteiro (p), que por sua vez guarda o endereço de x.

Quando o código executa `pp = 100`, a primeira desreferenciação (`*pp`) acessa o conteúdo de p, e a segunda (pp) segue a próxima seta para acessar e modificar diretamente o valor na "caixa" de x. Esse encadeamento cria duas setas consecutivas na memória (`pp -> p -> x`),

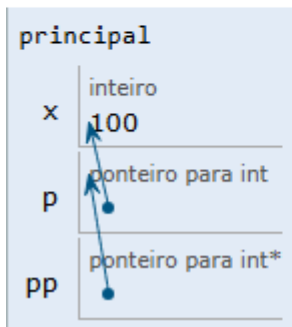
permitindo alterar o dado original através de dois níveis de indireção.


```

x = 42
*p = 42
**pp = 42
x apos **pp=100: 100

```

Pilha Monte



🔍 Exercício 10 — Heap vs. Stack

c

```

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 4;
    int *v = malloc(n * sizeof(int)); /* aloca no heap */

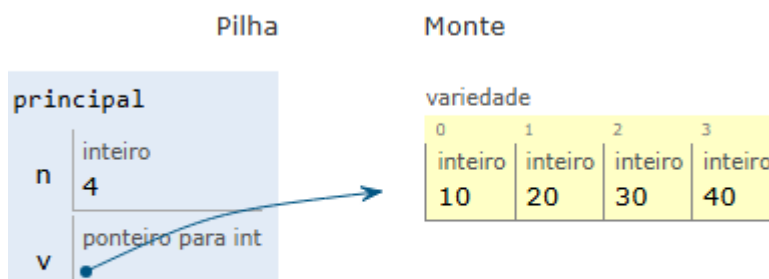
    for (int i = 0; i < n; i++)
        v[i] = (i + 1) * 10; /* 10 20 30 40 */

    for (int i = 0; i < n; i++)
        printf("v[%d] = %d\n", i, v[i]);

    free(v); /* devolve ao heap */
    v = NULL; /* evita uso acidental */
    return 0;
}

```

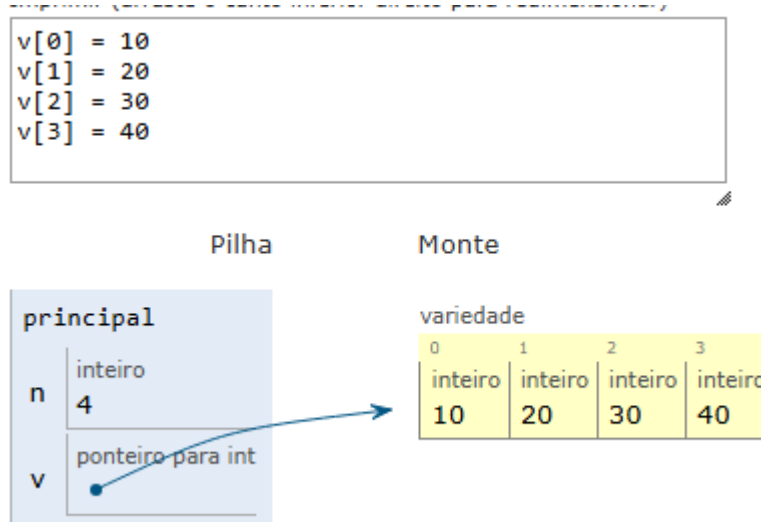
O que observar: no PythonTutor, o bloco alocado aparece no painel **Heap** (lado direito), separado do frame da stack. Após **free**, ele desaparece do heap.



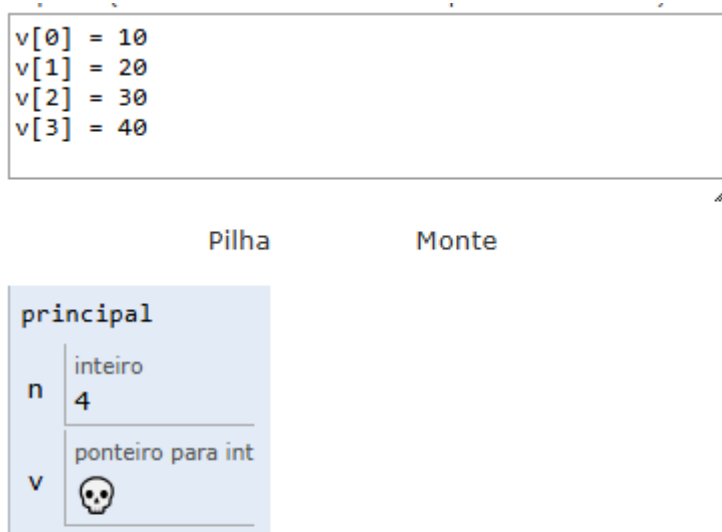
As variáveis locais da função main (como n e o ponteiro v) são guardadas

na Stack (pilha), que é gerenciada automaticamente pelo programa.

Ao chamar malloc, uma região de memória do tamanho solicitado é reservada dinamicamente no Heap (área de memória livre), e o ponteiro v na Stack apenas guarda o endereço para esse bloco.



Ao final, a função `free(v)` libera esse espaço de volta para o sistema, fazendo o bloco desaparecer do Heap, enquanto `v = NULL` garante que o ponteiro não tente acessar um endereço que não nos pertence mais.



Isso aconteceu porque o simulador tentou ler o ponteiro `v` depois que o `free(v)` já tinha deletado o bloco do Heap, ou o ponteiro foi usado antes de receber o endereço do malloc.

Nota: 🪦 significa que um ponteiro aponta para uma área de memória não alocada ou desalinhada com os limites dos dados. Os endereços indicados por 🪦 são *aproximados* e podem não corresponder ao endereço real do ponteiro. Selecione "visualização de dados em nível de byte" abaixo para ver mais detalhes:

Exercício 11a — Struct com acesso direto e via ponteiro

```

#include <stdio.h>

typedef struct {
    int x;
    int y;
} Ponto;

float distancia(Ponto *a, Ponto *b) {
    int dx = a->x - b->x;
    int dy = a->y - b->y;
    /* evitamos sqrt para simplificar */
    return (float)(dx*dx + dy*dy);
}

int main(void) {
    Ponto origem = {0, 0};
    Ponto p      = {3, 4};

    printf("p.x=%d p.y=%d\n", p.x, p.y);

    Ponto *ptr = &p;
    ptr->x = 10;
    printf("apos ptr->x=10: p.x=%d\n", p.x);

    printf("dist^2(origem,p) = %.0f\n", distancia(&origem, &p));
    return 0;
}

```

O que observar: `origem` e `p` como caixas compostas no frame; `ptr` como seta para `p`; `->` equivalendo a `(*ptr).x`.

O programa define a estrutura `Ponto` contendo dois membros inteiros, representados como caixas compostas contíguas na memória da Stack. O ponteiro `ptr` armazena o endereço da estrutura `p`, permitindo o uso do operador de seta (`->`) para modificar o campo `x` diretamente na memória, o que equivale sintaticamente a `(*ptr).x`. Essa mesma lógica de acesso por referência é aplicada ao chamar a função `distancia`, que recebe os endereços de origem e `p` para realizar os cálculos diretamente sobre as estruturas originais, evitando a duplicação desnecessária de dados na pilha.

```
p.x=3 p.y=4
apos ptr->x=10: p.x=10
```

Pilha

Monte

principal

sim

struct Ponto	
x	inteiro 0
y	inteiro 0

p

struct Ponto	
x	inteiro 10
y	inteiro 4

ptr

ponteiro para Ponto

sim

struct Ponto	
x	inteiro 0
y	inteiro 0

p

struct Ponto	
x	inteiro 10
y	inteiro 4

ptr

ponteiro para Ponto

distância

um

ponteiro para Ponto

b

ponteiro para Ponto

dx

inteiro
-10

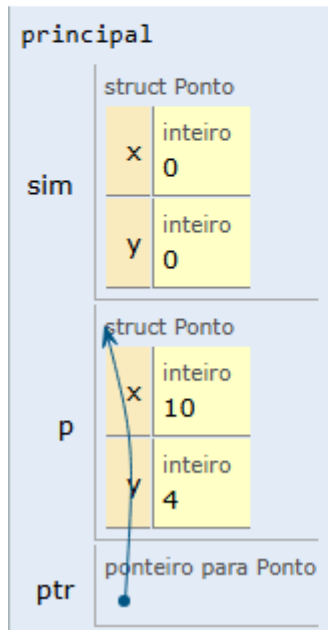
dy

inteiro
-4

```
p.x=3  p.y=4
apos ptr->x=10: p.x=10
dist^2(origem,p) = 116
```

Pilha

Monte



Exercício 11b — Union: membros compartilham memória

c

```
#include <stdio.h>
```

```
union Dado {
    int i;
    float f;
    char c;
};
```

```
int main(void) {
    union Dado d;
    d.i = 65;
    printf("d.i = %d\n", d.i);
    printf("d.c = %c\n", d.c); /* mesmo bytes, interpretado como char */
    d.f = 3.14f;
    printf("d.f = %.2f\n", d.f);
    /* d.i agora tem valor indefinido (sobrescrito por f) */
    return 0;
}
```

O que observar: o PythonTutor mostra um único bloco de memória; escrever em `d.f` sobrescreve o mesmo espaço de `d.i`.

```
d.i = 65
d.c = 'A'
```

```
d.i = 65
d.c = 'A'
d.f = 3.14
```

Pilha

Monte

Pilha

Monte

principal	
união Dado	
d	eu inteiro 65
	f flutuador 9,10844e-44
	c personagem 'UM'

principal	
união Dado	
d	eu inteiro 1078523331
	f flutuador 3.14
	c personagem '\xC3'

O programa ilustra como uma union aloca apenas um único bloco de memória compartilhado por todos os seus membros (i, f e c), diferentemente de uma struct. Ao atribuir 65 a `d.i`, o mesmo padrão de bits na memória é interpretado como o caractere 'A' ao ler `d.c` devido à tabela ASCII. No entanto, ao escrever o valor decimal 3.14f em `d.f`, o conteúdo anterior é completamente sobrescrito, demonstrando que a alteração de um membro invalida o valor dos outros por ocuparem o mesmo espaço físico.



Exercício 12 — Enum com switch

c

```
#include <stdio.h>
```

```
typedef enum { DOM=0, SEG, TER, QUA, QUI, SEX, SAB } DiaSemana;
```

```
const char *nome_dia(DiaSemana d) {
    switch (d) {
        case DOM: return "Domingo";
        case SEG: return "Segunda";
        case TER: return "Terca";
        case QUA: return "Quarta";
        case QUI: return "Quinta";
```

```

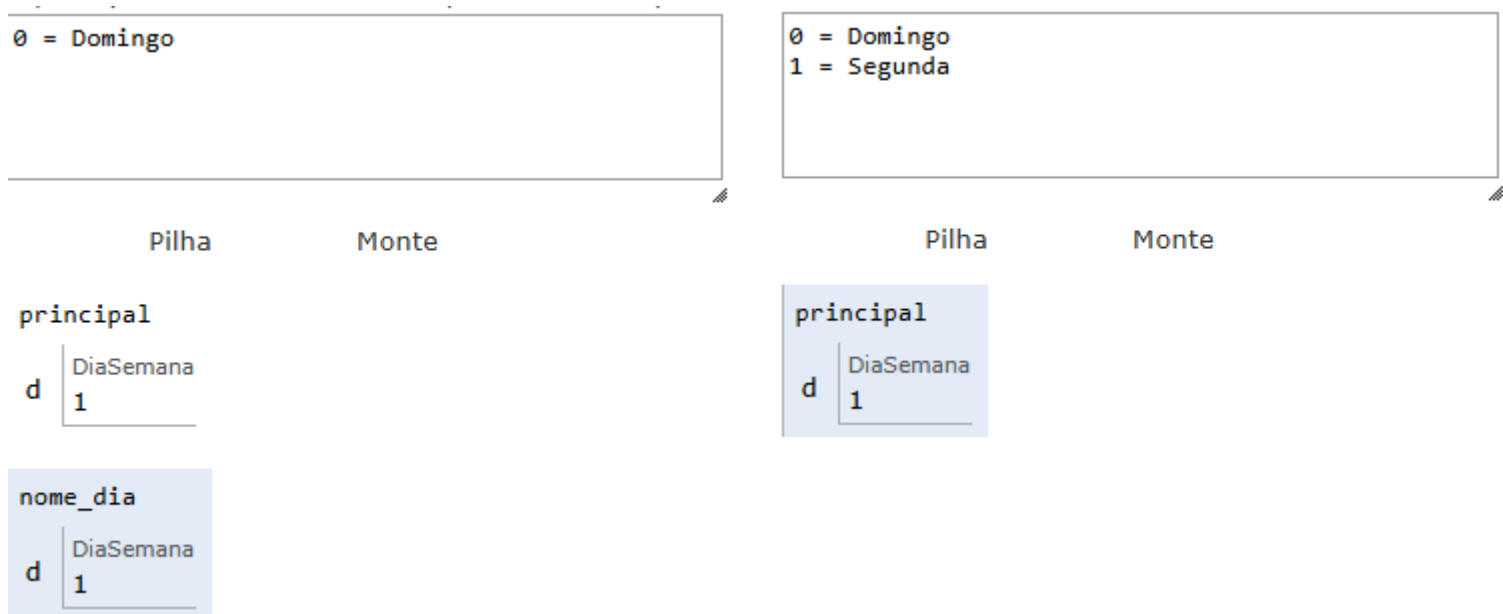
        case SEX: return "Sexta";
        case SAB: return "Sabado";
        default: return "???";
    }
}

int main(void) {
    for (DiaSemana d = DOM; d <= SAB; d++)
        printf("%d = %s\n", d, nome_dia(d));
    return 0;
}

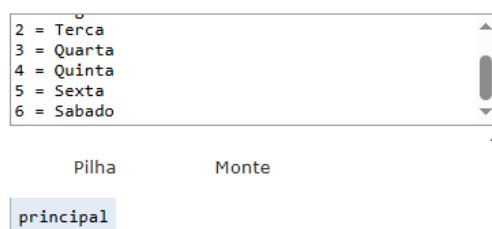
```

O que observar: enum é tratado como `int`; o laço `for` itera incrementando o valor numérico subjacente.

O programa demonstra que as constantes de uma enum são tratadas internamente pelo compilador C como números inteiros (int). Ao definir `DOM = 0`, os identificadores subsequentes (SEG, TER, etc.) recebem valores sequenciais incrementados automaticamente de 1 em 1 (0, 1, 2, etc.).



A função `nome_dia` aparece e some no painel da Stack porque, a cada iteração do laço, uma nova área de memória temporária (chamada stack frame) é criada para processar aquela chamada específica. Assim que a função atinge o comando `return` e devolve o texto para o `printf`, seu ciclo de vida termina e esse bloco é destruído e removido da memória imediatamente.



No laço for, a variável `d` do tipo `DiaSemana` é incrementada como um inteiro comum, permitindo que o bloco `switch` na função mapeie cada valor numérico à sua respectiva representação em texto.

Exercício 13 — Formatação de saída (sem `scanf` no PythonTutor)

 O PythonTutor não suporta entrada interativa via `scanf`. Use valores literais para testar a formatação.

```
c
#include <stdio.h>

int main(void) {
    int    i = 255;
    float  f = 3.14159f;
    char   c = 'Z';
    char   s[] = "Brasilia";

    printf("Decimal:   %d\n", i);
    printf("Hexadecimal: %x\n", i); /* ff */
    printf("Float 2dec: %.2f\n", f);
    printf("Char:      %c\n", c);
    printf("String:    %s\n", s);
    printf("Ponteiro:   %p\n", (void*)s);
    return 0;
}
```

O que observar: como `%x` mostra 255 como `ff`; `%p` mostra o endereço de `s`.

Hexadecimal:	ff
Float 2dec:	3.14
Char:	Z
String:	Brasilia
Ponteiro:	0xffff000bcf

Stack

Heap

main									
i	int 255								
f	float 3.14159								
c	char 'Z'								
s	array								
	0	1	2	3	4	5	6	7	8
	char	char	char	char	char	char	char	char	char
	'B'	'r'	'a'	's'	'i'	'l'	'i'	'a'	'\0'

O programa demonstra como diferentes especificadores de formatação na função printf interpretam e exibem os dados armazenados na memória. O especificador %x traduz o valor inteiro 255 para a sua representação hexadecimal ff, enquanto o %p exibe o endereço de memória inicial onde o array s ("Brasilia") está alocado na Stack. Além disso, o especificador %.2f limita a exibição do ponto flutuante a duas casas decimais, evidenciando como a linguagem C permite controlar a apresentação visual dos tipos de dados diretamente na saída padrão.



Exercício 14 — Constante e macro com parâmetro

c

```
#include <stdio.h>
```

```
#define PI 3.14159
```

```
#define QUADRADO(x) ((x)*(x))
```

```
#define MAX(a,b) ((a)>(b)?(a):(b))
```

```
int main(void) {
```

```
    double r = 5.0;
```

```
    printf("Area do circulo r=5: %.2f\n", PI * QUADRADO(r));
```

```
    int a = 7, b = 3;
```

```
    printf("MAX(%d,%d) = %d\n", a, b, MAX(a,b));
```

```

/* armadilha classica de macro sem parenteses */
printf("QUADRADO(a+1) = %d\n", QUADRADO(a+1)); /* (a+1)*(a+1) = 64 */
return 0;
}

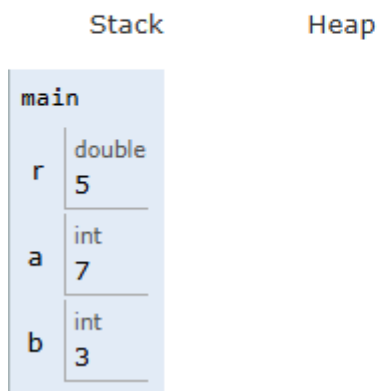
```

O que observar: macros são substituição textual — o PythonTutor mostra o código **após** a expansão do pré-processador. Tente `MAX(a++, b)` para ver o efeito colateral.

```

Area do circulo r=5: 78.54
MAX(7,3) = 7
QUADRADO(a+1) = 64

```



O programa demonstra o funcionamento das diretivas de pré-processamento (`#define`), que realizam uma substituição puramente textual no código-fonte antes de sua compilação real. Ao executar o código, o compilador não enxerga as chamadas de funções para as macros, mas sim as expressões expandidas diretamente no corpo da função `main`.

O uso rigoroso de parênteses na macro `QUADRADO(x)` como `((x)*(x))` é fundamental para evitar problemas de precedência de operadores, garantindo que a expressão `QUADRADO(a+1)` se expanda de forma segura para `((a+1)*(a+1))` (resultando em 64) em vez de `a+1*a+1` (que resultaria em 15).

Além disso, o comportamento de substituição textual traz riscos de efeitos colaterais: ao passar um argumento com incremento, como em `MAX(a++, b)`, a expressão se expande para `((a++)>(b)?(a++):(b))`, fazendo com que a variável `a` seja avaliada e incrementada duas vezes na memória caso ela seja o maior valor.

Exercício 15a — Escopo de bloco

c

```
#include <stdio.h>
```

```
int main(void) {
    int x = 1;
}
```

```

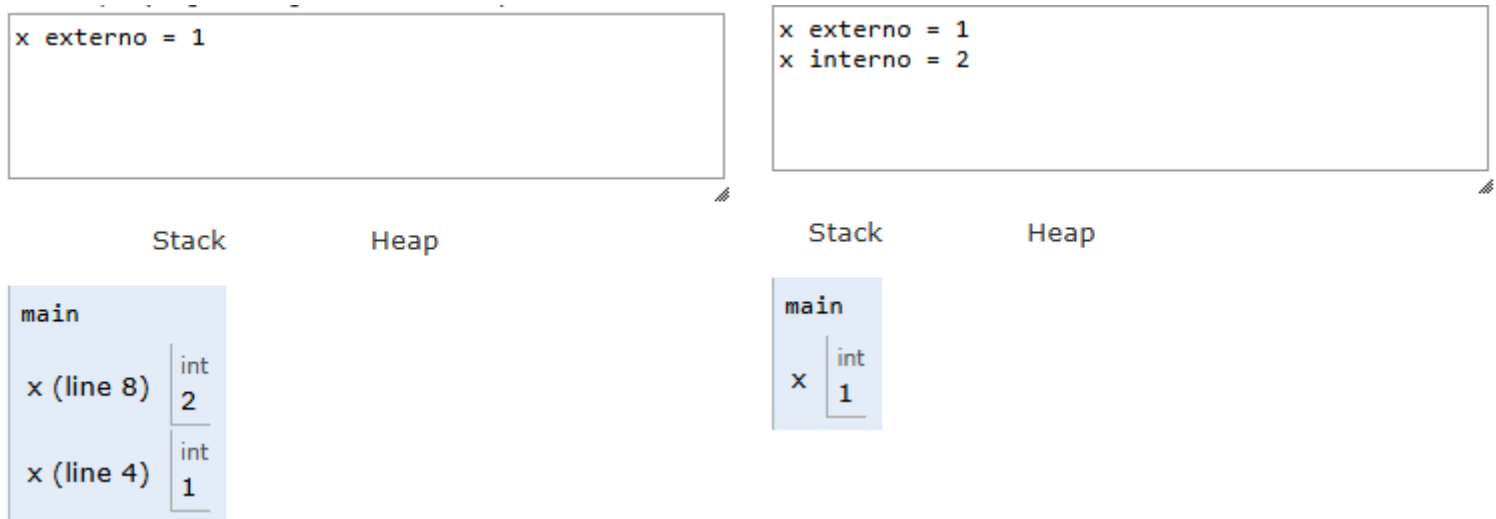
printf("x externo = %d\n", x);

{
    int x = 2; /* nova variavel, esconde a externa */
    printf("x interno = %d\n", x);
}

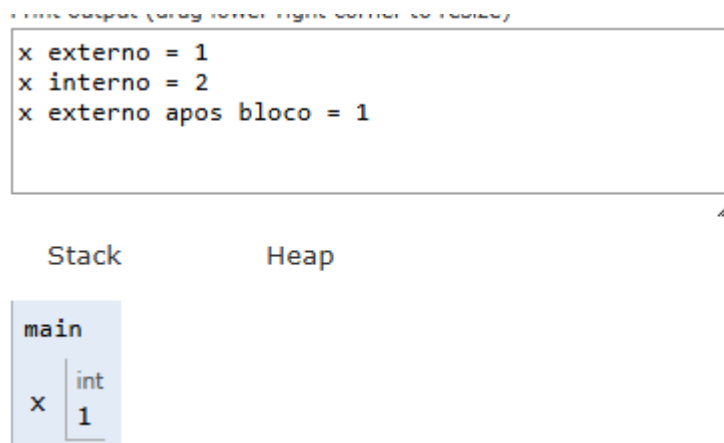
printf("x externo apos bloco = %d\n", x); /* ainda 1 */
return 0;
}

```

O que observar: o `x` interno aparece no frame apenas enquanto o bloco `{ }` está ativo; ao sair, o frame volta a enxergar o `x` externo.



O programa demonstra o conceito de escopo de bloco e sobreamento (shadowing) de variáveis em C. Ao abrir um bloco delimitado por chaves `{ ... }`, uma nova variável `x` (com valor 2) é alocada temporariamente na Stack, ocultando temporariamente a variável homônima do escopo externo. Assim que o fluxo de execução sai desse bloco, o `x` interno é destruído e removido da pilha, permitindo que o programa volte a acessar apenas o `x` original (com valor 1).





Exercício 15b — Variável **static** local (persiste entre chamadas)

c

```
#include <stdio.h>

void contador(void) {
    static int n = 0; /* inicializado UMA VEZ; persiste */
    n++;
    printf("chamada numero %d\n", n);
}

int main(void) {
    contador(); /* 1 */
    contador(); /* 2 */
    contador(); /* 3 */
    return 0;
}
```

O que observar: `n` não é recriado a cada chamada — o PythonTutor o mostra como variável **global** (fora do frame), mesmo sendo declarado dentro da função.

chamada numero 1

chamada numero 1
chamada numero 2
chamada numero 3

Stack

Heap

Stack

Heap

main

main

contador

n [static]

int
1

O programa demonstra o comportamento de uma variável local static, que é inicializada apenas uma vez na primeira chamada da função e preserva seu valor entre execuções subsequentes. Ao contrário de variáveis automáticas que são criadas e destruídas na Stack, a variável `n` é alocada na região de dados estáticos da memória, persistindo durante toda a execução do programa. Por esse motivo, o simulador a exibe fora dos frames temporários das funções, permitindo que seu valor seja incrementado de forma cumulativa a cada chamada.

Exercício 16 — Detectando estouro de buffer (simulação segura)

c

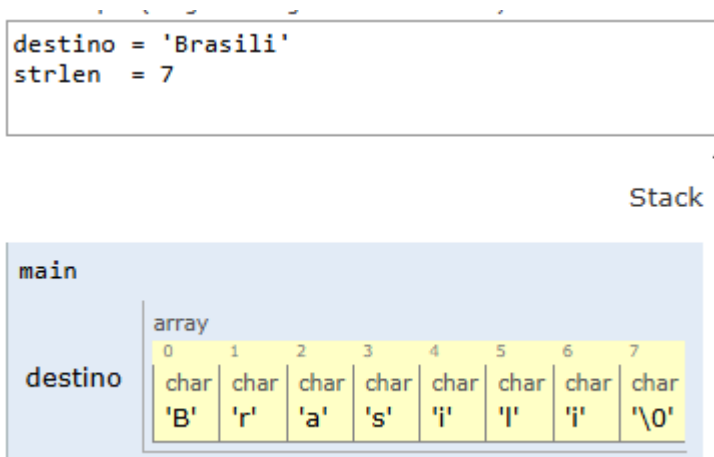
```
#include <stdio.h>
#include <string.h>

int main(void) {
    char destino[8];

    /* SEGURO: limita a copia ao tamanho do buffer */
    strncpy(destino, "Brasilia-DF", sizeof(destino) - 1);
    destino[sizeof(destino) - 1] = '\0'; /* garante terminador */

    printf("destino = '%s'\n", destino);
    printf("strlen = %zu\n", strlen(destino));
    return 0;
}
```

O que observar: `destino` tem 8 bytes; `strncpy` copia no máximo 7 caracteres + `'\0'`. No PythonTutor, veja como o array é preenchido byte a byte e truncado.



O programa demonstra como prevenir um buffer overflow (estouro de buffer) limitando rigorosamente a cópia de dados para um array de tamanho fixo. Ao utilizar a função `strncpy` com o limite de `sizeof(destino) - 1` (7 bytes), o código copia apenas os primeiros sete caracteres ("Brasili") da string "Brasilia-DF", impedindo que os dados excedentes invadam outras regiões de memória na Stack. A definição manual do caractere nulo (`\0`) na última posição do array garante que a string seja finalizada de forma válida, resultando em uma truncagem segura e controlada.